

TRABAJO PRÁCTICO OBLIGATORIO

Programación II

Profesor/as:

Nicolas Ignacio Perez

Integrantes del Grupo Nro 1:

<i>Valentino Ariotti</i>	-1144115
<i>Bruno Lautaro Bianchi</i>	-1146454
<i>Francisco Espinel</i>	-1114625
<i>Patricio Ivan Funes</i>	-1170506
<i>Valentin Marcos Oubiña</i>	-1138710

Cuatrimestre: 1 – Año: 2025



**UNIVERSIDAD ARGENTINA DE LA EMPRESA
FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS**

Planteo del problema:

Durante nuestra investigación sobre problemas reales que pueden ser resueltos con los algoritmos vistos en clase encontramos que, hablando de los servicios de emergencia urbana, es muy importante optimizar el tiempo de respuesta de las ambulancias. Para modelar esta situación empleamos un grafo dirigido en el que cada vértice representa una intersección o punto de interés (por ejemplo, “Hospital” o “Paciente”) y cada arista conecta dos vértices adyacentes, cargando un peso igual al tiempo promedio de recorrido entre ellos (medido en minutos).

El objetivo es, dado un vértice origen y un vértice, determinar la ruta cuya suma total de pesos sea mínima, garantizando así el camino más corto posible. Este problema de camino más corto en grafos ponderados es esencial en logística y transporte, y su solución directa impacta en la eficiencia y la vida de las personas dependientes de la atención médica urgente.

Descripción del algoritmo:

Para resolver este problema empleamos el algoritmo de Dijkstra, un método que construye iterativamente un conjunto de vértices D con las distancias mínimas confirmadas desde el origen. Inicialmente, se asigna a cada vértice adyacente al origen su peso de arista y a los demás el valor infinito; a continuación, en cada iteración se selecciona el vértice no procesado con la distancia más pequeña, se añade a D y se actualizan las distancias de sus vecinos si se descubre un camino más corto pasando por él. El proceso se repite hasta incluir todos los vértices. Gracias al uso de una cola de prioridad basada en un montículo binario, la complejidad temporal queda en $O((V + E) \cdot \log V)$ y el espacio adicional en $O(V + E)$.

Implementación bajo TDA:

Para garantizar una solución modular y testeable, definimos primero la interfaz genérica `Grafo<V>`, que permite añadir vértices y aristas, así como consultar el conjunto de vértices y los vecinos de cualquiera de ellos. A partir de esta interfaz, se crea la clase `GrafoListaAdy<V>`, que implementa la representación mediante listas de adyacencia usando un `Map<V, Map<V, Double>>`. El algoritmo de Dijkstra se encapsula en la clase `Dijkstra<V>`, cuyo método `shortestPaths` recibe cualquier implementación de `Grafo<V>` y retorna un mapa con las distancias mínimas desde el vértice origen hasta todos los demás. El código de referencia, disponible en el repositorio del curso, mantiene esta separación de responsabilidades para facilitar futuros cambios de representación sin afectar la lógica del algoritmo.

Ejecución y resultados:

Para ilustrar su funcionamiento, tomamos un grafo de ejemplo con seis vértices etiquetados $V1 \dots V6$ y aristas con pesos tal como se presenta en los materiales de la cátedra. Tras inicializar las distancias ($\text{dist}[V1]=0$, resto a infinito) y progresar paso a paso escogiendo siempre el vértice con menor distancia provisional y relajando sus

aristas, se obtiene al final un vector de distancias mínimas desde V1 que coincide con los resultados mostrados en el ejemplo de clase, confirmando que aplicamos la lógica de Dijkstra bien.

Pruebas bajo TDA:

La robustez del sistema se verifica mediante tres grupos principales. Primero, se comprueba la correcta construcción del grafo: después de añadir vértices y aristas, estos aparecen en la colección y los pesos son los esperados. Segundo, se ponen a prueba casos triviales, como un grafo de un solo vértice (distancia cero) o la falta de aristas (distancias infinitas entre nodos distintos). Finalmente, se diseñan tests de rutas sencillas, por ejemplo, un grafo de dos vértices con una arista de peso 5.0. Para asegurar que el algoritmo retorna exactamente 5.0 para el destino e infinito para nodos no conectados.

Otros algoritmos:

Aunque **Dijkstra** es ideal para grafos con pesos no negativos, existen alternativas útiles en distintos escenarios. El algoritmo de **Floyd** utiliza programación dinámica para comparar todos los posibles caminos entre cada par de vértices, iterando sobre una matriz de distancias y actualizándola según la fórmula $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$, lo que resulta en una complejidad $O(V^3)$ y es especialmente adecuado para grafos densos o cuando se requieren caminos cortos entre todos los pares. Por su parte, **Kruskal** construye un árbol de expansión mínimo (MST) en grafos conexos y ponderados ordenando las aristas por peso y aplicando una estructura de conjuntos disjuntos (union-find), con complejidad $O(E \cdot \log V)$, lo que lo hace muy eficiente para encontrar un MST pero no para el problema de caminos más cortos.